ViT 论文问答记录

配套《ViT 论文总结》使用。按论文章节组织；CNN 部分是阅读本文所需的预备知识，论文正文没有展开讲。

---

§0 预备知识：CNN（论文外）

Q：卷积到底在做什么？

卷积核(convolution kernel，也叫 filter)是一个很小的权重方块，一般 3×3。它盖在图像的某个 3×3 区域上，把对应位置的 9 个像素值和 9 个权重**逐个相乘再求和**（不是矩阵乘法，数学上是两个小方块的内积；实现里为了效率会把图像展开成矩阵真的用矩阵乘法算，那是工程手段），得到一个数；然后向右滑一格重复，滑完整张图得到一张 feature map（特征图）。

做卷积的理由有两个。一是单个像素值几乎不携带信息，有意义的东西藏在一小片区域内像素之间的关系里（边缘、条纹、色块），卷积把"一个点和它周围的点"一起送进一次运算。二是对照不用卷积的做法：把 224×224×3 展平成长度 150528 的向量接全连接层，参数量约 2×10^{10}，而且展平后"相邻"这件事就没了——左上角的像素和它正下方的像素在向量里隔了 224×3 个位置。卷积可以看成**一个被约束过的全连接层**：只允许连接局部 3×3 邻域（其余权重强制为 0），且所有位置复用同一组权重(weight sharing)。一个 3×3、64→64 通道的卷积核参数量约 3.7 万，小六个数量级。

Q：用 3×3 卷积核，图像是不是被缩小 9 倍？再做一层缩小 81 倍？

不是。相邻两次滑动是**重叠**的，不是把图切成互不相交的小块。输出边长 = 输入边长 − 核边长 + 1，224 做一层 3×3 得到 222；再在四周补一圈零(padding)就能保持 224 不变。**卷积本身不负责缩小图像**，做一百层尺寸也可以一直是 224×224。

实际网络里图像变小来自另外两个刻意加的操作：stride（步长，核每次挪 2 格而不是 1 格，边长减半）和 pooling（池化，把特征图切成互不重叠的 2×2 只保留最大值或平均值，边长减半）。而且每次空间尺寸减半时通道数一般翻倍，是在拿分辨率换特征种类。

Q：卷积核一般多大？一般做多少层？

主流是 **3×3**。VGG（2014）之后定型：两层 3×3 的感受野等于一层 5×5，但参数更少（9+9 vs 25）、中间多一次非线性。例外有两个：网络最前面那层有时用 7×7 stride 2（ResNet 就是这样，一上来把 224 变 112），AlexNet 甚至用过 11×11 stride 4；以及 **1×1** 卷积，不看任何邻域，纯粹用来改通道数。

层数十几到一百多，常见档位就是 ResNet 后面的数字：ResNet-18/34/50/101/152。注意这个数字是**带权重的层总数**（含最后的线性分类层），不是"多少层卷积"，而且 ResNet 的结构单位是残差块。论文里的 ResNet152x4 是 152 层、每层通道数加宽 4 倍，"x2""x3"都是宽度倍数不是层数。

Q：什么是通道?

channel（通道）就是**同一个空间位置上并列存放的多个数**。彩色图每个像素是 R、G、B 三个数，所以形状是 224×224×3，这个 3 就是输入通道数（论文里的 C）。

经过一个有 64 个核的卷积层后，每个核产出一张特征图，叠起来输出 224×224×64，每个位置上的 64 个数表示"匹配第 1 种到第 64 种模式的程度"。所以**输出通道数 = 这一层卷积核的个数**，是自己定的超参数；RGB 的含义是人规定的，这 64 个通道的含义是训练学出来的。

通道多不等于"更精细"——精细是空间分辨率(H、W)的事，通道多是**种类多**。

Q：卷积核到底是 3×3 还是 3×3×通道数？哪些权重不一样?

核的真实形状是 3×3×C_in：它盖上去时同时吃掉所有输入通道那一小片，全部乘完加起来才得到一个数。

不一样的是两个维度：**通道之间**不一样（一个 3×3×64 的核在 64 个输入通道上用 64 个不同的 3×3 权重块）；**核与核之间**不一样（这一层 64 个核是 64 套互不相同的权重）。

位置之间是一样的——同一个核在所有空间位置用完全同一组权重，这就是 weight sharing。一层卷积的权重张量形状是 (C_out, C_in, 3, 3)，**没有任何一维是空间位置**；空间位置只出现在数据张量 (N, C, H, W) 里。参数量 = 3×3×C_in×C_out。

Q：CNN 之后怎么出分类结果?

卷积层堆完得到比如 7×7×2048 的特征图，标准做法是 **GAP（Global Average Pooling，全局平均池化）**：每个通道的 7×7 个数求平均塌成一个数，得到长度 2048 的向量。GAP 只负责把空间维压掉，**后面还要接一个线性层**把 2048 维映射到类别数，才轮到 softmax。训练是普通监督学习：交叉熵 loss、反向传播、SGD 或 Adam，和其他神经网络没有本质区别。

另外每层卷积后面跟的是固定搭配的归一化和非线性激活（conv → BN → ReLU），没有非线性的话堆多少层都等价于一层。

Q：CNN 现在还有人用吗，还是都转向 VLM 了?

这两者不是同一层级：CNN 是 backbone（把图像变成特征的那一段），VLM 是一类系统，它内部同样需要 backbone。和 CNN 构成替代关系的是 **ViT**，不是 VLM。

在需要大规模预训练、追求最强表示的场景里 ViT 已经是默认选择（CLIP、SigLIP 等视觉编码器都是 ViT）。CNN 仍是首选的地方：医学影像分割的 U-Net、实时检测（YOLO）、端侧模型（算子成熟、延迟可控）、数据量小又无合适预训练权重的垂直领域；扩散模型长期用 U-Net。以及混合结构（论文 2.7 节的 hybrid 是雏形）。

Q：CNN 的全称?

Convolutional Neural Network，卷积神经网络。简称也写作 ConvNet。kernel 和 filter 在这个语境下是同一样东西。

---

§1 Introduction

Q：ViT 的全称?

Vision Transformer。论文标题 "An Image is Worth 16x16 Words" 里没出现这个词，标题是同一件事的另一种讲法。

模型变体记号：ViT-B/16 = Base + 16×16 图块，ViT-L/16 = Large，ViT-H/14 = Huge + 14×14 图块。**斜杠后面是 patch size，不是层数。**

---

§3.1 Method — Vision Transformer

Q：Figure 1 里的 "Linear Projection of Flattened Patches" 是不是就是一个线性层?

是，就是 `nn.Linear`，没有特殊之处。输入是展平的图块，长度 $P^2 \cdot C = 16 \times 16 \times 3 = 768$；权重 E 形状 768×D；输出长度 D。ViT-B 的 D 正好是 768 所以看不出在做什么，ViT-L 的 D=1024，这层就是 768→1024。

它对应文本里的 word embedding：文本那边是查表（token id → 取嵌入矩阵第 id 行），图像这边没有词表可查（图块取值连续），所以改成一次矩阵乘法投影到 D 维。196 个图块共用同一个 E。这一步等价于一个 kernel 16×16、stride 16、输出通道 D 的卷积层，官方实现就是用 `nn.Conv2d` 写的。

Q：分类词元是什么？是"分类训练"的意思吗?

不是训练方式，就是一个**向量**。`x_class` 是长度 D 的可学习参数，整个模型只有这一个（不是每张图一个），前向时拼在 196 个图块嵌入最前面，序列变成 197。

作用是当"收集器"：它本身不携带任何图像内容（对每张图都是同一个初始值），在自注意力里唯一能做的就是从 196 个图块聚合信息。走完 L 层后它的输出 z^0_L 被当作整张图的表示 y，分类头只接在它上面，其余 196 个位置的输出在分类时丢掉。

"词元"只是 token 的中译，叫 token 是因为它在序列里占一个位置、平等参与注意力，与文本无关。

Q：现在还用分类词元吗?

主流已经不用了，更多算是"尽量不改原版 Transformer"留下的历史痕迹。后来的视觉编码器大多改用 GAP 或 attention pooling。论文 D.3 已经埋了伏笔：作者试过 GAP 一开始效果极差，查下来既不是那个多出来的 token 的问题、也不是 GAP 的问题，**纯粹是两者需要的学习率不同**，调对之后效果相当。既然一样，后来就没理由再多带一个特殊 token。

（VLM 里这个问题形态变了：视觉编码器的输出要把所有图块表示都送进语言模型当视觉 token，"把整张图压成一个向量"这件事本身就不太出现了。）

Q：位置嵌入是不是和 Transformer 的完全一样?

机制一样：形状 (N+1)×D，按位置逐个**加**到 token 上（$z_0 = [x_class;\ x^1_pE;\ \dots] + E_pos$），不是拼接。

但有一处不同：原版 Transformer 用的是固定的正弦函数编码，算出来就不动；ViT 用的是**可学习**的位置嵌入（这点与 BERT 一致）。

"1D"是相对"2D"说的：14×14 的图块本有行列坐标，但 ViT 不管，按 raster order 编号成 0…195 当一维序列处理。D.4 消融：有无位置嵌入差距很大（ImageNet 5-shot 线性 0.614 → 0.642），但 1D / 2D / 相对位置嵌入**之间**几乎没差别；作者解释是图块级输入只有 14×14，学这点空间关系对哪种编码方式都不难。

Q：什么是"可学习的位置嵌入"?

把正弦公式换成一张**参数表**：建一个 (N+1)×D 的张量随机初始化，注册成模型参数，跟着反向传播一起更新，前向时第 i 个 token 取第 i 行加上去。梯度会流到它上面，因为它直接参与了加法。

```
self.pos_embed = nn.Parameter(torch.randn(1, N + 1, D) * 0.02)   # 可学习
x = x + self.pos_embed
```

正弦版本则是 `register_buffer`：随模型保存和搬设备，但不进优化器、不接收梯度。

两个要点。一是**它是参数表不是函数，长度写死**：训练时 197 行，来了第 198 个位置就没有对应的行——这正是 3.2 节要插值的根源。二是**它学到的东西确实有结构**：初始化时 197 个向量彼此无关，训练后 Figure 7（中）显示位置越近的图块位置嵌入余弦相似度越高、同行同列明显更高，模型完全从数据里学出了"哪些图块在空间上挨着"。

---

§3.2 Method — 微调与更高分辨率

Q：换分辨率、插值位置嵌入，这具体是怎么做的?

预训练输入 224×224、图块 16×16，N = 196，加分类词元共 197 个 token。微调换成 384×384（Table 4；Table 2 的 ImageNet 那几行用了 512 和 518），图块大小仍是 16，N = 24×24 = 576，序列 577。

关键一问：哪些东西会因序列变长而失效？**只有位置嵌入**。线性投影 E 是 $(P^2 \cdot C) \times D$，逐图块作用；注意力的 U_qkv 是 D×3D_h；LayerNorm 和 MLP 也逐位置作用——全都与 N 无关。这就是"Transformer 能处理任意序列长度（只受显存限制）"的含义。唯独 `E_pos` 是 (N+1)×D 的参数表，只存了 197 行，现在需要 577 行。

插值就是把缺的 380 行造出来：把预训练那 196 行按 raster order 排回 14×14 网格，每格一个 D 维向量——此时形状 14×14×D，和一张 14×14 像素、D 通道的图片同构；然后当图片放大到 24×24。新网格第 j 个位置对应旧坐标 j×13/23，除两端外都是小数（j=1 → 0.565，j=2 → 1.130…），取相邻整数位置的向量按距离加权平均，二维就是四个邻居一起加权。**论文原文只写了"做二维插值"，没指定算法**；实现里一般用双线性或双三次。分类词元那一行不参与，因为它没有空间位置，原样保留。

被放大的是**参数**不是数据——图像是真的换成 384×384 的原图、切出 576 个真实图块。这里也**没有任何掩码**，位置嵌入是加上去的；mask 在本文只出现在 4.6 节的 masked patch prediction。

这么插之所以合理，正是因为 Figure 7（中）那个证据：相邻位置的嵌入本来就彼此接近，中间值才有意义。而因为插值是**按图块在原图中的行列坐标**做的（人手动告诉模型第 196 行参数对应右下角），这一步就是把"图块排成二维网格"这个知识重新塞进模型——所以论文说：切图块和这个分辨率调整，是全模型里**仅有的两处**人工注入二维先验的地方。

两个同属 3.2 的细节：微调时预训练的预测头被丢掉，换成零初始化的 D×K 线性层（B.1.1 补充：整个头两层全换掉，比只重新初始化最后一层更稳）；代价是序列 197 → 577 而自注意力开销随长度平方增长，所以只在微调阶段这么干。

---

§4 Experiments

Q：为什么要微调？这个微调在做什么，对应 LLM 的哪一部分?

做的事：拿预训练好的 ViT 权重当起点，换掉分类头，在下游数据集上**继续训练全部参数**（带动量的 SGD、批大小 512、2500–20000 步，Table 4）。

需要它是因为预训练和下游任务目标对不上：ViT 在 JFT 上学的是 18k 个内部类别，下游是 CIFAR-100 的 100 类、Pets 的 37 类，类别体系都不一样，预训练的分类头只能扔掉重建；编码器权重虽通用也要在新分布上再调。4.2 节那些数字全部是"预训练一次、微调七次"的结果。

对应 LLM 就是 **SFT 的位置**，即"预训练 → 微调"两段式范式的第二段，论文 1 节讲 NLP 现状时说的就是这件事。但要注意时代差异：2020 年的"微调"指 BERT 那种换任务头、全参数更新，与现在后训练的 SFT + RLHF 不是一回事——这里没有指令、没有偏好、没有 RL，就是普通监督分类。

论文还有一个并列口径叫 **few-shot linear**（4.1 Metrics）：**冻结**编码器，只把表示拿去解带正则的最小二乘回归，对应 LLM 里的 linear probing。主要结果用微调准确率，few-shot 线性只在微调太贵时用（Figure 4、D.2 用的就是它）。

Q：SOTA 是什么?

state of the art，当前最好水平。论文正文写全称（4.2 节标题 "Comparison to State of the Art"），SOTA 是社区通用缩写。

它指某个 benchmark 上目前公开报告过的最高成绩，会随时间被刷新，不是某个固定模型，所以说"某模型是 SOTA"要带上任务。本文 4.2 节写明了当时的分工：ImageNet 上是 Noisy Student，其余数据集上是 BiT-L——Table 2 里那两列就是这两个对手。

复现可行性（3090 单卡，论文外）

论文核心实验：不可能。 ViT-L/16 在 JFT-300M 上预训练是 0.68k TPUv3 核·天，ViT-H/14 是 2.5k；最便宜的 ImageNet-21k + ViT-L/16 论文说是 8 核 TPUv3 约 30 天，换算到单卡是年的量级。而且 JFT-300M 从未公开。训练配置 batch size 4096，24GB 放不下，要梯度累积模拟。

微调实验：可做，最值得先做。 官方放出了预训练权重（第 1 页脚注），timm 可直接加载。ViT-B/16 的 ImageNet-21k 权重微调 CIFAR-100，Table 5 报 91.67，有机会接近。224 分辨率 batch 64 舒服；384 时序列 197→577 显存吃紧，配 AMP + 梯度累积。CIFAR-100 微调 10000 步是几小时到一天；Pets、Flowers 只要 500 步。这条路验证"预训练表示强"，但验证不了核心论点。

核心论点的小规模复现：这才有意思。 论文真正的主张是 4.3 节的"数据规模决定归纳偏置是帮助还是限制"。从零训练一个小 ViT 和参数量相当的 ResNet，在 CIFAR-10（5 万张）上对比，大概率能复现"ViT 明显打不过 ResNet"，即 Figure 3 最左端那个点；再用 ImageNet-1k 子集逐级放大看两条曲线差距如何缩小。单卡能跑。注意尺度差三个数量级，只能看趋势不能看绝对数字；从零训 ViT 在小数据上对超参和正则化极其敏感，论文 4.3 自己也说为小数据集专门调了权重衰减、dropout、标签平滑三个参数。